

LAPORAN TUGAS PERTEMUAN 2
PROGRAMMING LANGUAGE PRAGMATICS
ANALISIS PARADIGMA PEMROGRAMAN UNTUK LIMA STUDI
KASUS

Laporan Ini Disusun Untuk Memenuhi Tugas Kuliah
Mata Kuliah : Programming Language Pragmatics



Disusun Oleh :

Nama : Ghaisan Khoirul Badruzaman

NIM : 251524048

Kelas : 2B-D4

PROGRAM STUDI D4 TEKNIK INFORMATIKA
JURUSAN TEKNIK KOMPUTER DAN INFORMATIKA
POLITEKNIK NEGERI BANDUNG
2026

PENDAHULUAN

Tugas pertemuan 2 meminta analisis pendekatan pemrograman untuk lima studi kasus di slide. Setiap kasus dijawab dengan tabel sepuluh aspek dari slide Tugas, mulai dari karakteristik masalah, konsep komputasi yang dominan, pendekatan yang dipilih beserta alasannya, contoh struktur, kelebihan dan keterbatasan, risiko terhadap kualitas software, karakteristik ISO/IEC 25010 yang relevan, sampai perlu tidaknya kombinasi pendekatan.

Pilihan pendekatan merujuk ke tabel paradigma di slide 2, yaitu imperative, object-oriented, functional, logic, concurrent, dan event-driven. Contoh struktur untuk setiap kasus ditulis sebagai potongan kode yang benar-benar dijalankan, lengkap dengan perintah dan outputnya, supaya contohnya bisa dicek ulang dan tidak berhenti di pseudocode. Penulisan ISO/IEC 25010 mengikuti versi 2023 seperti tugas sebelumnya: nama karakteristik dalam bahasa Inggris dan sub-karakteristiknya di dalam kurung.

KASUS 1: SISTEM KERANJANG BELANJA E-COMMERCE

Perusahaan e-commerce butuh fitur keranjang belanja untuk aplikasi web dan mobile, tempat setiap pengguna menyimpan beberapa produk lengkap dengan kode, nama, kategori, harga, dan jumlah yang dibeli. Setiap kali produk ditambah atau jumlahnya diubah, subtotal harus ikut diperbarui, lalu saat checkout sistem menghitung total belanja, potongan harga, pajak, ongkos kirim, sampai total pembayaran. Sistem juga harus menolak kondisi yang tidak valid, seperti produk yang tidak tersedia, jumlah yang melebihi stok, dan kode promo yang sudah tidak berlaku, sekaligus memberi potongan kepada pengguna yang berhak, padahal aturan-aturan ini bisa diganti perusahaan kapan saja. Semua hasil itu harus tetap benar walaupun banyak pengguna menambah produk, mengubah jumlah, checkout, dan memperbarui stok pada waktu yang hampir bersamaan.

Pendekatan utama: **Object-Oriented (Java)**.

Tabel 1. Analisis Kasus 1

Aspek Analisis	Jawaban
Karakteristik masalah	Masalahnya campuran. Ada data yang strukturnya jelas, yaitu produk (kode, nama, kategori, harga) dan keranjang tiap pengguna yang berisi item beserta jumlahnya. Ada juga aturan bisnis yang bisa diganti perusahaan kapan saja: potongan harga bersyarat, kode promo yang punya masa berlaku, dan hak potongan untuk pengguna tertentu. Checkout sendiri berupa perhitungan berurutan dari subtotal sampai total bayar, sedangkan stok adalah data bersama yang bisa diubah banyak pengguna web dan mobile pada saat yang hampir bersamaan.
Konsep komputasi yang dominan	State yang berubah karena aksi pengguna: jumlah item di keranjang naik atau turun saat produk ditambah atau jumlahnya diubah, dan stok berkurang setiap ada checkout yang berhasil. Selain itu ada transformasi data berurutan saat checkout (subtotal, potongan, PPN, ongkir, total) yang sebenarnya tidak perlu mengubah data apa pun. Stok juga diakses bersamaan, jadi cek lalu potong stok harus atomik supaya dua pembeli tidak sama-sama lolos untuk barang terakhir, yaitu race condition yang berujung overselling.
Pendekatan pemrograman yang dipilih	Object-Oriented dengan Java (JDK 25) sebagai pendekatan utama, misalnya di backend Spring Boot yang melayani aplikasi web dan mobile. Data dibuat sebagai record (<code>Produk</code> , <code>Item</code> , <code>Tagihan</code>), keranjang berupa daftar <code>Item</code> yang jumlahnya diperbarui setiap produk yang sama ditambah lagi, aturan potongan berupa objek yang mengimplementasikan interface <code>Diskon</code> , dan aktivitas pengguna menjadi method pada objek <code>Toko</code> seperti <code>checkout</code> . Di dalamnya dipakai dua paradigma lain dari tabel slide: Functional untuk perhitungan checkout lewat fungsi murni <code>hitung</code> , dan Concurrent untuk pemotongan stok yang harus atomik.

Aspek Analisis	Jawaban
Alasan pemilihan berdasarkan karakteristiknya	Isi kasusnya memang berupa benda yang punya data sekaligus perilaku: produk punya harga, item punya subtotal, promo punya tanggal berakhir, dan toko punya stok. Dengan enkapsulasi, objek menjaga aturannya sendiri, misalnya <code>Item</code> langsung menolak jumlah kurang dari 1 dan field <code>stok</code> dibuat <code>private</code> sehingga hanya bisa dipotong lewat method <code>Toko</code> . Aturan yang sering berubah ditangani dengan polimorfisme: promo atau hak potongan baru cukup ditulis sebagai kelas baru yang mengimplementasikan <code>Diskon</code> , tanpa menyentuh kode checkout. Java juga umum dipakai untuk backend e-commerce dan sudah punya dukungan thread serta kunci bawaan untuk melayani banyak pengguna.
Contoh struktur/pseudocode	<code>Record Produk</code> , <code>Item</code> , dan <code>Tagihan</code> menyimpan data, interface <code>Diskon</code> diisi record <code>Promo</code> yang punya tanggal berakhir dan lambda <code>member</code> untuk hak potongan 5%, lalu <code>hitung</code> menghitung subtotal, potongan, PPN 11%, ongkir, dan total tanpa mengubah data apa pun. Method <code>checkout</code> yang <code>synchronized</code> mengecek semua item dulu baru memotong stok, dan pengecekan yang sama menolak produk yang tidak tersedia (stok 0) maupun jumlah yang melebihi stok. Di output, dari tiga pengguna yang checkout bersamaan dengan stok buku 3, satu berhasil, satu ditolak karena promo <code>MERDEKA17</code> kedaluwarsa, satu ditolak karena stok kurang, dan stok buku berakhir di 1. Potongan kodenya ada di Kode 1 di bawah tabel ini.
Kelebihan	Nama kelas dan method-nya sama dengan istilah bisnis, jadi kodenya gampang dibaca dan dibagi ke beberapa anggota tim. Aturan diskon bisa ditambah atau diganti tanpa mengubah <code>hitung</code> maupun <code>checkout</code> . Karena <code>hitung</code> murni, tiap skenario tagihan bisa diuji dengan unit test biasa, dan karena stok hanya berubah di satu tempat yang dikunci, overselling bisa dicegah.
Keterbatasan	Contoh ini memakai satu kunci untuk seluruh objek <code>Toko</code> , jadi checkout produk yang berbeda pun tetap antri satu per satu, dan <code>synchronized</code> hanya berlaku di dalam satu JVM. Begitu backend berjalan di beberapa server, stok harus dijaga database, misalnya lewat transaksi atau <code>UPDATE ... WHERE stok >= jumlah</code> . Aturan yang saling terkait, misalnya promo yang tidak boleh digabung dengan potongan member, juga belum tertangani oleh daftar <code>Diskon</code> yang sekadar dijumlahkan. Kode Java pun cukup panjang untuk hal kecil, dan desain OOP gampang berakhir menjadi satu kelas besar yang mengurus semuanya.
Risiko terhadap kualitas software	Risiko terbesar ada di stok. Kalau ada satu jalur yang mengubah stok tanpa kunci, misalnya fitur admin untuk memperbarui stok, race condition bisa membuat dua pembeli sama-sama membayar barang terakhir padahal hanya satu yang bisa dikirim. Kalau syarat promo diselipkan langsung sebagai if di dalam <code>checkout</code> , setiap perubahan kebijakan berisiko merusak perhitungan total. Kunci yang terlalu besar juga bisa membuat checkout lambat saat flash sale, dan keranjang yang diubah dari web dan mobile sekaligus bisa menampilkan jumlah barang yang berbeda kalau tidak disinkronkan.

Aspek Analisis	Jawaban
ISO/IEC 25010 yang relevan	Functional Suitability (functional correctness): total, potongan, PPN, dan ongkir harus tepat, promo kedaluwarsa tidak boleh terpakai, dan nilai rupiah disimpan sebagai long, bukan double. Reliability (faultlessness): checkout dan stok tetap benar walaupun banyak pengguna bertransaksi bersamaan, tanpa overselling dan tanpa stok yang terpotong sebagian. Maintainability (modifiability, testability): aturan promo dan hak potongan mengikuti kebijakan yang berubah, jadi aturan dipisah ke <code>Diskon</code> dan <code>hitung</code> dibuat murni supaya mudah diuji. Performance Efficiency (time behaviour, capacity): checkout harus tetap cepat saat trafik tinggi, sedangkan kunci yang terlalu besar membatasi jumlah transaksi yang bisa dilayani.
Apakah perlu kombinasi pendekatan?	Ya. OOP menjadi kerangka utama untuk entitas produk, keranjang, promo, dan toko, sedangkan perhitungan checkout ditulis dengan gaya Functional: <code>hitung</code> menerima item, aturan, dan tanggal, lalu mengembalikan <code>Tagihan</code> tanpa efek samping. Concurrent dipakai di pemotongan stok, di contoh dengan <code>synchronized</code> dan di sistem nyata dengan transaksi database per produk. Event-driven dipakai di aplikasi web dan mobile, karena klik tambah produk, ubah jumlah, dan checkout adalah event yang ditangani handler lalu diteruskan ke backend. Kalau aturan promo makin banyak dan saling terkait, bagian aturan bisa dipindah ke rule engine yang gayanya mirip Logic programming.

Kode 1. Keranjang belanja: record data, aturan Diskon, fungsi murni, dan stok yang dikunci (Java, berkas Toko.java)

```
import java.time.LocalDate;
import java.util.*;
record Produk(String kode, String nama, String kategori, long harga) {}
record Item(Produk produk, int jumlah) {
    Item { if (jumlah < 1) throw new Ditolak("jumlah minimal 1"); }
    long subtotal() { return produk.harga() * jumlah; }
}
record Tagihan(long belanja, long diskon, long ppn, long ongkir, long total) {}
class Ditolak extends RuntimeException { Ditolak(String m) { super(m); } }
interface Diskon { long potongan(long belanja, LocalDate tgl); }
record Promo(String kode, long nilai, LocalDate akhir) implements Diskon {
    public long potongan(long belanja, LocalDate tgl) {
        if (tgl.isAfter(akhir)) throw new Ditolak(kode + " kedaluwarsa");
        return nilai;
    }
}
public class Toko {
    private final Map<String, Integer> stok = new HashMap<>();
    // fungsi murni: hasilnya hanya bergantung pada input
    static Tagihan hitung(List<Item> isi, List<Diskon> aturan, LocalDate tgl) {
        long belanja = isi.stream().mapToLong(Item::subtotal).sum();
        long diskon = Math.min(belanja,
            aturan.stream().mapToLong(d -> d.potongan(belanja, tgl)).sum());
        long netto = belanja - diskon, ppn = netto * 11 / 100, ongkir = 20_000;
        return new Tagihan(belanja, diskon, ppn, ongkir, netto + ppn + ongkir);
    }
    synchronized void checkout(List<Item> isi, List<Diskon> aturan) {
        try {
            Tagihan t = hitung(isi, aturan, LocalDate.now());
            for (Item it : isi) // semua item dicek dulu, baru stok dipotong
                if (it.jumlah() > stok.getOrDefault(it.produk().kode(), 0))
```

```

        throw new Ditolak("stok " + it.produk().nama() + " kurang");
    }
    for (Item it : isi)
        stok.merge(it.produk().kode(), -it.jumlah(), Integer::sum);
    System.out.println(t);
} catch (Ditolak e) { System.out.println("gagal: " + e.getMessage()); }
}

public static void main(String[] args) {
    var toko = new Toko();
    toko.stok.putAll(Map.of("P01", 5, "P02", 3));
    var kaos = new Produk("P01", "Kaos Polos", "Pakaian", 85_000);
    var buku = new Produk("P02", "Buku Java", "Buku", 120_000);
    Diskon member = (belanja, tgl) -> belanja * 5 / 100; // hak potongan
    var promo = new Promo("MERDEKA17", 50_000, LocalDate.of(2026, 8, 31));
    var isi = List.of(new Item(buku, 2), new Item(kaos, 1));
    List.of(List.of(member, promo), List.of(member), List.of(member))
        .parallelStream().forEach(aturan -> toko.checkout(isi, aturan));
    System.out.println("stok akhir " + toko.stok);
}
}

```

Dijalankan dengan perintah `java Toko.java`, outputnya:

```

Tagihan[belanja=325000, diskon=16250, ppn=33962, ongkir=20000, total=362712]
gagal: MERDEKA17 kedaluwarsa
gagal: stok Buku Java kurang
stok akhir {P01=4, P02=1}

```

KASUS 2: SISTEM INFORMASI RUMAH SAKIT

Rumah sakit membutuhkan sistem untuk mengelola pasien, dokter, jadwal pemeriksaan, rekam medis, resep, dan pembayaran. Satu pasien bisa punya banyak kunjungan, dan hasil setiap pemeriksaan (diagnosis, tindakan, resep) harus tersimpan di rekam medisnya. Petugas pendaftaran, dokter, dan bagian administrasi punya hak akses berbeda, termasuk aturan bahwa dokter hanya boleh memperbarui rekam medis pasien yang ia tangani. Sistem juga harus siap menerima jenis tenaga kesehatan, layanan, metode pembayaran, dan integrasi laboratorium baru tanpa perlu ditulis ulang.

Pendekatan utama: **Object-Oriented (Java)**.

Tabel 2. Analisis Kasus 2

Aspek Analisis	Jawaban
Karakteristik masalah	Masalah ini berisi banyak entitas yang punya data dan perilaku sendiri: pasien dengan identitasnya, dokter dengan spesialisasinya, jadwal pemeriksaan, rekam medis, resep, dan pembayaran. Relasinya banyak yang berbentuk satu-ke-banyak, misalnya satu pasien punya banyak kunjungan dan satu dokter menangani banyak pasien. Rekam medis bersifat sensitif, jadi petugas pendaftaran, dokter, dan bagian administrasi harus punya hak akses yang berbeda. Kebutuhannya juga akan terus bertambah, mulai dari jenis tenaga kesehatan, layanan, dan metode pembayaran baru sampai integrasi dengan sistem laboratorium.
Konsep komputasi yang dominan	Sistem ini bekerja dengan objek yang menyimpan state lalu saling memanggil method, sesuai pertanyaan What objects interact? pada tabel paradigma. Setiap kejadian di rumah sakit mengubah state objek tertentu: pendaftaran menambah jadwal pemeriksaan, pemeriksaan menambah satu <i>Kunjungan</i> berisi diagnosis, tindakan, dan resep ke rekam medis pasien, dan pelunasan mengubah status pembayaran. Relasi antarobjek, misalnya dokter dengan pasien yang ditanganinya, ikut menentukan siapa boleh melakukan apa. Karena itu abstraksi utamanya adalah object dan encapsulation, yaitu data rekam medis dibungkus di dalam objek dan hanya berubah lewat method yang memeriksa hak akses.
Pendekatan pemrograman yang dipilih	Object-Oriented , diimplementasikan dengan Java. Setiap entitas dijadikan kelas yang membungkus data dan perilakunya, misalnya <i>Pasien</i> menyimpan riwayat kunjungan dan menyediakan method <i>catat</i> untuk menambah rekam medis. Peran pengguna beserta daftar haknya dibuat sebagai <i>enum</i> sehingga tabel RBAC terkumpul di satu tempat, sedangkan bagian yang akan bertambah seperti metode pembayaran dan integrasi laboratorium dibuat sebagai interface. Java dipilih karena class, interface, dan access modifier seperti <i>private</i> tersedia langsung di bahasanya, dan kesalahan tipe sudah tertangkap saat kompilasi.

Aspek Analisis	Jawaban
Alasan pemilihan berdasarkan karakteristiknya	Rekam medis yang sensitif paling aman dilindungi dengan encapsulation: daftar kunjungan dibuat <code>private</code> dan hanya bertambah lewat method yang memeriksa hak akses. Relasi satu-ke-banyak dimodelkan dengan composition (<code>Pasien</code> memiliki daftar <code>Kunjungan</code>), begitu juga peran (<code>Pengguna</code> memiliki <code>Peran</code>) supaya staf yang berganti peran cukup diubah datanya, bukan kelasnya. Inheritance hanya dipakai untuk hubungan is-a yang membawa aturan tambahan, seperti <code>Dokter</code> yang dibatasi pada pasien yang ditanganinya. Kebutuhan yang bertambah ditangani dengan polymorphism: tenaga kesehatan dengan aturan khusus cukup menjadi subclass baru dari <code>Pengguna</code> , dan metode pembayaran baru tinggal mengimplementasikan interface <code>MetodeBayar</code> , sehingga kode pemanggilnya tidak perlu diubah.
Contoh struktur/pseudocode	Potongan kode Java di bawah menyimpan aturan RBAC di enum <code>Peran</code> (pendaftaran: data pasien dan jadwal, dokter: rekam medis, administrasi: pembayaran), lalu <code>Dokter</code> meng-override method <code>boleh</code> untuk menambah syarat bahwa pasien harus ada di daftar <code>ditangani</code> . Rekam medis hanya bertambah lewat <code>Pasien.catat</code> dan hanya dibaca lewat <code>riwayat()</code> yang mengembalikan salinan read-only, sehingga catatan dr. Sari tersimpan sedangkan percobaan dr. Andi (dokter lain) dan Rina (petugas pendaftaran) ditolak. Pembayaran dan integrasi laboratorium tidak dimasukkan supaya potongan tetap pendek, tetapi keduanya memakai polymorphism yang sama lewat interface <code>MetodeBayar</code> dan <code>LayananLab</code> . Potongan kodenya ada di Kode 2 di bawah tabel ini.
Kelebihan	Struktur kelasnya mengikuti cara kerja rumah sakit (pasien, dokter, kunjungan, pembayaran), sehingga mudah dibahas bersama pihak rumah sakit dan mudah dipahami anggota tim baru. Aturan akses terkumpul di dua tempat, yaitu tabel <code>Peran</code> dan override <code>boleh</code> pada <code>Dokter</code> , jadi tidak tersebar di setiap layar aplikasi. Karena rekam medis hanya bisa diubah lewat <code>catat</code> , bagian program lain tidak bisa menghapus atau menimpa riwayat kunjungan diam-diam. Metode pembayaran baru seperti QRIS atau tenaga kesehatan baru seperti apoteker cukup ditambah sebagai kelas atau peran baru, dan setiap kelas bisa diuji terpisah dengan unit test.
Keterbatasan	Kalau inheritance dipakai berlebihan, hierarki kelas menjadi kaku, misalnya sulit memodelkan satu orang yang merangkap dokter dan kepala poli. Objek Java hanya hidup di memori, jadi rekam medis perlu dipetakan ke tabel basis data, dan pemetaan ini menambah kode serta celah bug. Java juga cenderung panjang karena satu entitas kecil sudah butuh field, constructor, dan beberapa method. Selain itu, OOP tidak mengatur akses bersamaan, misalnya dua petugas mengubah jadwal pemeriksaan yang sama pada waktu yang sama, sehingga tetap perlu transaksi di basis data.
Risiko terhadap kualitas software	Risiko terbesar ada di sisi keamanan: method baru yang lupa memanggil <code>boleh</code> , atau getter yang mengembalikan list asli alih-alih salinan, bisa membuat rekam medis dibaca atau diubah oleh pihak yang tidak berhak. Kelas pusat yang terus ditemplei fitur dapat berubah menjadi god class yang sulit dianalisis dan diuji. Jika kelas domain langsung memanggil API laboratorium tertentu, pergantian vendor lab akan memaksa perubahan di banyak tempat. Memuat seluruh riwayat kunjungan pasien lama sekaligus ke memori juga dapat memperlambat waktu respons.

Aspek Analisis	Jawaban
ISO/IEC 25010 yang relevan	Security (confidentiality, integrity, accountability) paling utama, karena rekam medis hanya boleh dibuka dokter yang menangani, hanya berubah lewat <code>catat</code> , dan setiap kunjungan menyimpan nama dokter pengisinya. Maintainability (modularity, modifiability, testability) terkait kebutuhan rumah sakit yang terus bertambah: setiap entitas ada di kelas sendiri, dan aturan seperti penolakan dr. Andi bisa diuji dengan unit test. Flexibility (adaptability, replaceability) dibutuhkan agar layanan dan metode pembayaran baru bisa masuk, serta vendor laboratorium bisa diganti tanpa membongkar kelas domain. Compatibility (interoperability) relevan karena permintaan tes harus terkirim ke sistem laboratorium dan hasilnya kembali masuk ke rekam medis pasien yang benar.
Apakah perlu kombinasi pendekatan?	Ya. OOP tetap menjadi kerangka utama untuk entitas dan hak akses, tetapi integrasi laboratorium lebih cocok memakai pendekatan event-driven: hasil tes yang dikirim lab diperlakukan sebagai event, lalu handler menambahkannya ke rekam medis dan memberi tahu dokter yang menangani. Penyimpanan dan pencarian riwayat kunjungan serta data pembayaran diserahkan ke SQL di basis data relasional yang bersifat deklaratif. Untuk rekap bagian administrasi, misalnya total pembayaran per metode dalam sebulan, gaya functional dengan Java Stream lebih ringkas dan tidak mengubah data asli.

Kode 2. Struktur kelas dan hak akses rekam medis (Java, berkas `RumahSakit.java`)

```
import java.util.*;

enum Hak { DATA_PASIEN, JADWAL, REKAM_MEDIS, PEMBAYARAN }
enum Peran { // RBAC: daftar hak melekat ke peran, bukan ke orangnya
    PENDAFTARAN(Hak.DATA_PASIEN, Hak.JADWAL), DOKTER(Hak.REKAM_MEDIS),
    ADMINISTRASI(Hak.PEMBAYARAN);
    final Set<Hak> hak;
    Peran(Hak... h) { hak = Set.of(h); }
}

class Pengguna {
    final String nama; final Peran peran;
    Pengguna(String n, Peran p) { nama = n; peran = p; }
    boolean boleh(Hak h, Pasien p) { return peran.hak.contains(h); }
}

class Dokter extends Pengguna {
    final String spesialisasi;
    private final Set<Pasien> ditangani = new HashSet<>();
    Dokter(String n, String sp) { super(n, Peran.DOKTER); spesialisasi = sp; }
    void tangani(Pasien p) { ditangani.add(p); }
    @Override boolean boleh(Hak h, Pasien p) { // syarat tambahan untuk dokter
        return super.boleh(h, p) && ditangani.contains(p);
    }
}

record Kunjungan(String dokter, String diagnosis, String tindakan,
    String resep) {}

class Pasien {
    final String nama;
    private final List<Kunjungan> riwayat = new ArrayList<>();
    Pasien(String nama) { this.nama = nama; }
    void catat(Pengguna u, String diagnosis, String tindakan, String resep) {
        if (!u.boleh(Hak.REKAM_MEDIS, this)) throw new SecurityException(
            u.nama + " (" + u.peran + ") ditolak mengubah RM " + nama);
    }
}
```

```

        riwayat.add(new Kunjungan(u.nama, diagnosis, tindakan, resep));
    }
    List<Kunjungan> riwayat() { return List.copyOf(riwayat); } // read-only
}

public class RumahSakit {
    public static void main(String[] args) {
        Pasien budi = new Pasien("Budi");
        Dokter sari = new Dokter("dr. Sari", "Penyakit Dalam");
        sari.tangani(budi); // hasil jadwal yang dibuat petugas pendaftaran
        for (Pengguna u : List.of(sari, new Dokter("dr. Andi", "Anak"),
            new Pengguna("Rina", Peran.PENDAFTARAN))) {
            try { budi.catat(u, "Hipertensi", "EKG", "Amlodipin 5 mg"); }
            catch (SecurityException e) { System.out.println(e.getMessage()); }
        }
        for (Kunjungan k : budi.riwayat())
            System.out.println("RM Budi: " + k.dokter() + ", " + k.diagnosis()
                + ", " + k.tindakan() + ", " + k.resep());
    }
}

```

Dijalankan dengan perintah `java RumahSakit.java`, outputnya:

```

dr. Andi (DOKTER) ditolak mengubah RM Budi
Rina (PENDAFTARAN) ditolak mengubah RM Budi
RM Budi: dr. Sari, Hipertensi, EKG, Amlodipin 5 mg

```

KASUS 3: ANALISIS DATA PENJUALAN

Perusahaan memiliki data transaksi penjualan selama setahun yang mencatat nomor transaksi, tanggal, produk, kategori, harga, jumlah, lokasi, dan pelanggan. Dari data itu manajemen ingin memperoleh total penjualan, rata-rata nilai transaksi, produk dengan penjualan tertinggi, total per kategori, penjualan per wilayah, dan daftar transaksi dengan kriteria tertentu, termasuk laporan gabungan seperti transaksi produk tertentu yang dihitung nilainya, dikelompokkan per kategori, lalu diurutkan. Karena data bertambah setiap bulan, operasi penyaringan, pemetaan, pengelompokan, transformasi, dan agregasi harus mudah dirangkai ulang untuk analisis baru tanpa mengubah data asli. Solusinya juga dinilai dari keterbacaan kode, kemudahan pengujian, dan kemampuan mengolah data dalam jumlah besar.

Pendekatan utama: **Functional (Haskell)**.

Tabel 3. Analisis Kasus 3

Aspek Analisis	Jawaban
Karakteristik masalah	Datanya berupa kumpulan transaksi satu tahun dengan atribut tetap, yaitu nomor transaksi, tanggal, produk, kategori, harga, jumlah, lokasi, dan pelanggan. Data ini terus bertambah setiap bulan, tetapi proses analisis hanya membacanya dan tidak boleh mengubahnya. Yang berganti-ganti adalah pertanyaan manajemen: total penjualan, rata-rata nilai transaksi, produk dengan penjualan tertinggi, total per kategori, penjualan per wilayah, dan daftar transaksi dengan kriteria tertentu. Semua laporan itu tersusun dari langkah yang sama, yaitu saring, hitung nilai, kelompokkan, jumlahkan, lalu urutkan, hanya kombinasi dan kriterianya yang berbeda. Tidak ada interaksi antarobjek atau kejadian yang harus ditanggapi seketika, jadi inti masalahnya adalah transformasi data yang harus tetap berjalan saat volumenya membesar.
Konsep komputasi yang dominan	Pengolahan di kasus ini berupa transformasi koleksi data lewat fungsi, sesuai pertanyaan "What transformation is applied?" pada tabel paradigma. Kebutuhan manajemen terpetakan langsung ke higher-order function: <code>filter</code> untuk transaksi produk, wilayah, atau kriteria tertentu, <code>map</code> untuk menghitung nilai harga kali jumlah, <code>M.fromListWith</code> untuk mengelompokkan per produk, kategori, dan wilayah, <code>sortOn</code> untuk peringkat, dan <code>fold (foldl ')</code> untuk total dan rata-rata. Laporan baru dibentuk dengan komposisi fungsi, sementara data transaksi diperlakukan sebagai nilai immutable yang hanya dibaca.
Pendekatan pemrograman yang dipilih	Paradigma Functional dari tabel slide dengan bahasa Haskell (GHC 9.10, dijalankan lewat <code>runghc</code>) yang hanya memakai pustaka bawaan GHC, yaitu <code>Data.List</code> untuk <code>foldl'</code> dan <code>sortOn</code> serta <code>Data.Map.Strict</code> dari paket <code>containers</code> untuk pengelompokan. Setiap transaksi dimodelkan sebagai record <code>Transaksi</code> yang immutable, setiap langkah analisis ditulis sebagai fungsi murni kecil seperti <code>nilai</code> , <code>total</code> , <code>kelompokkan</code> , dan <code>peringkat</code> , lalu setiap laporan dirangkai dengan operator komposisi <code>..</code> . Pada sistem sebenarnya, pembacaan file transaksi dan pencetakan laporan dibatasi di <code>main</code> yang berjalan di monad IO, sehingga inti analisis bebas efek samping.

Aspek Analisis	Jawaban
Alasan pemilihan berdasarkan karakteristiknya	Analisis hanya membaca transaksi lalu menghasilkan ringkasan atau daftar baru, sehingga fungsi murni cocok: input yang sama selalu menghasilkan laporan yang sama, dan data asli aman karena nilai di Haskell tidak bisa ditimpa. Permintaan "transaksi produk tertentu, hitung nilainya, kelompokkan per kategori, lalu urutkan" bisa ditulis hampir kata per kata sebagai <code>peringkat kategori . filter (...)</code> . Penjumlahan bersifat asosiatif dan komutatif, jadi total per kategori atau wilayah tiap bulan bisa dihitung terpisah lalu digabung dengan <code>M.unionWith (+)</code> , cocok untuk data yang bertambah tiap bulan dan untuk pemrosesan paralel. Dengan gaya imperative, setiap laporan baru butuh loop dan variabel akumulator sendiri yang rawan salah dan sulit dipakai ulang.
Contoh struktur/pseudocode	Potongan Haskell di bawah memodelkan transaksi sebagai record <code>Transaksi</code> dan menyusun fungsi kecil <code>nilai</code> , <code>total</code> , <code>rataRata</code> , <code>kelompokkan</code> , dan <code>peringkat</code> yang dipakai ulang untuk sudut pandang produk, kategori, dan wilayah, sedangkan daftar transaksi dengan kriteria tertentu cukup <code>filter kriteria</code> . Laporan kombinasi <code>laporanProduk</code> hanya merangkai fungsi yang sudah ada dengan komposisi. Dua baris output terakhir menunjukkan bahwa ringkasan lama yang digabung dengan ringkasan April sama dengan hasil hitung ulang, dan <code>dataTrx</code> tetap berisi 5 transaksi. Potongan kodenya ada di Kode 3 di bawah tabel ini.
Kelebihan	Setiap laporan cukup satu baris komposisi yang dibaca dari kanan ke kiri sebagai urutan langkah, sehingga sudut pandang baru dari manajemen dilayani dengan merangkai fungsi yang sudah ada tanpa mengubah fungsi lama. Fungsi murni seperti <code>total</code> dan <code>kelompokkan</code> bisa diuji hanya dengan list berisi beberapa transaksi, tanpa database atau mock, dan sifat seperti "Lama+April = hitung ulang" bisa diuji otomatis dengan QuickCheck. Karena tidak ada state bersama yang diubah, ringkasan per bulan atau per wilayah bisa dihitung paralel tanpa race condition. Kalau transaksi dibaca dari file secara lazy dan hanya dilewati sekali, <code>total</code> dan <code>kelompokkan</code> mengolahnya sambil dibaca, sehingga memori hanya terpakai untuk hasil ringkasan, bukan seluruh transaksi.
Keterbatasan	Lazy evaluation bisa menimbulkan space leak: <code>rataRata</code> pada contoh memanggil <code>total</code> lalu <code>length</code> , sehingga seluruh list tertahan di memori sampai lintasan kedua selesai, dan untuk data setahun perlu diganti fold strict satu lintasan. Mengurutkan semua transaksi butuh seluruh data sekaligus dan tidak bisa di-streaming, jadi kalau data tidak muat di RAM, pekerjaan itu diserahkan ke database atau Spark. Tiap langkah juga membuat list atau Map baru, sehingga alokasi dan kerja garbage collector bisa lebih besar dibanding loop imperative yang memperbarui satu hash map di tempat. Ekosistem Haskell untuk grafik dan ekspor laporan ke Excel jauh lebih kecil dibanding Python, dan tim perlu waktu untuk terbiasa dengan monad.
Risiko terhadap kualitas software	Pipeline yang benar untuk data contoh bisa lambat atau kehabisan memori pada data akhir tahun karena space leak atau karena seluruh transaksi dimuat sekaligus, dan masalah ini baru terlihat saat volume sudah besar. Mengambil elemen pertama dari peringkat yang kosong dengan fungsi parsial, misalnya untuk bulan yang belum punya transaksi, membuat laporan crash saat runtime, karena itu contoh memakai <code>take 1</code> . Kunci pengelompokan berupa String rawan salah ketik, misalnya "Bandung" dan "bandung" terhitung dua wilayah, dan transaksi tidak valid seperti harga negatif atau nomor transaksi ganda ikut terjumlah diam-diam kalau tidak ada validasi sebelum agregasi. Rantai komposisi point-free yang terlalu panjang juga menurunkan keterbacaan bagi anggota tim baru.

Aspek Analisis	Jawaban
ISO/IEC 25010 yang relevan	Maintainability (modularity, reusability, analysability, modifiability, testability): tantangan kasus meminta operasi penyaringan sampai agregasi yang jelas, dapat digunakan kembali, mudah dibaca, dan mudah diuji, sedangkan analisis baru cukup dibuat dengan merangkai fungsi yang sudah ada. Performance Efficiency (time behaviour, resource utilization, capacity): laporan harus tetap cepat dan hemat memori walau transaksi bertambah tiap bulan, dan kapasitasnya cukup untuk data satu tahun penuh. Flexibility (scalability): ringkasan yang bisa dihitung per bulan lalu digabung memungkinkan beban dipecah ke beberapa core atau mesin saat data membesar. Security (integrity): proses analisis tidak boleh mengubah data transaksi asli, dan immutability mencegah perubahan itu terjadi karena kesalahan kode.
Apakah perlu kombinasi pendekatan?	Ya. Inti analisis tetap functional, tetapi membaca file atau tabel transaksi, menulis laporan, dan menjalankan pembaruan tiap bulan adalah efek samping, sehingga ditulis dengan gaya imperative di lapisan luar (<code>main</code> dalam monad IO), pola yang dikenal sebagai functional core, imperative shell. Saat data sudah besar, pendekatan concurrent dipakai dengan memecah transaksi per bulan atau per wilayah menjadi task yang berjalan bersamaan di beberapa core atau mesin, misalnya dengan library parallel di Haskell atau cluster Spark yang memakai pola map dan reduce yang sama, lalu hasil tiap task digabung dengan <code>M.unionWith (+)</code> . Penyaringan awal dan penyimpanan transaksi bisa diserahkan ke database lewat query SQL yang bersifat deklaratif.

Kode 3. Pipeline analisis penjualan dengan fungsi murni (Haskell, berkas `Penjualan.hs`)

```
import Data.List (foldl', sortOn)
import qualified Data.Map.Strict as M

data Transaksi = Transaksi
  { noTrx :: Int, tanggal, produk, kategori :: String
  , harga, jumlah :: Int, lokasi, pelanggan :: String }

nilai :: Transaksi -> Int
nilai t = harga t * jumlah t

total, rataRata :: [Transaksi] -> Int
total = foldl' (+) 0 . map nilai
rataRata ts = if null ts then 0 else total ts `div` length ts -- dua lintasan

kelompokkan :: (Transaksi -> String) -> [Transaksi] -> M.Map String Int
kelompokkan kunci = M.fromListWith (+) . map (\t -> (kunci t, nilai t))

peringkat :: (Transaksi -> String) -> [Transaksi] -> [(String, Int)]
peringkat kunci = sortOn (negate . snd) . M.toList . kelompokkan kunci

-- laporan baru cukup merangkai fungsi yang sudah ada
laporanProduk :: [String] -> [Transaksi] -> [(String, Int)]
laporanProduk ps = peringkat kategori . filter ((`elem` ps) . produk)

dataTrx, april :: [Transaksi]
dataTrx =
  [ Transaksi 1 "2026-01-05" "Laptop" "Elektronik" 7500000 2 "Bandung" "Andi"
  , Transaksi 2 "2026-01-09" "Mouse" "Aksesoris" 150000 10 "Jakarta" "Sari"
  , Transaksi 3 "2026-02-11" "Pulpen" "ATK" 5000 200 "Bandung" "Budi"
  , Transaksi 4 "2026-02-14" "Laptop" "Elektronik" 7500000 1 "Surabaya" "Sari"
```

```

    , Transaksi 5 "2026-03-02" "Keyboard" "Aksesoris" 350000 4 "Bandung" "Dewi" ]
    april = [Transaksi 6 "2026-04-03" "Mouse" "Aksesoris" 150000 5 "Jakarta" "Rina"]

main :: IO ()
main = do
    tampil "Total penjualan" (total dataTrx)
    tampil "Rata-rata transaksi" (rataRata dataTrx)
    tampil "Produk tertinggi" (take 1 (peringkat produk dataTrx))
    tampil "Per kategori" (peringkat kategori dataTrx)
    tampil "Per wilayah" (peringkat lokasi dataTrx)
    tampil "Laptop, Mouse" (laporanProduk ["Laptop", "Mouse"] dataTrx)
    tampil "Trx Bandung > 1 juta" (map noTrx (filter kriteria dataTrx))
    -- April masuk sebagai list baru, ringkasan lama tinggal digabung
    tampil "Lama+April = hitung ulang" (gabung == perKategori (dataTrx ++ april))
    tampil "Data asli" (length dataTrx, total dataTrx)
    where
        tampil label x = putStrLn (label ++ ": " ++ show x)
        kriteria t = lokasi t == "Bandung" && nilai t > 1000000
        perKategori = kelompokkan kategori
        gabung = M.unionWith (+) (perKategori dataTrx) (perKategori april)

```

Dijalankan dengan perintah `runghc Penjualan.hs`, outputnya:

```

Total penjualan: 26400000
Rata-rata transaksi: 5280000
Produk tertinggi: [("Laptop",22500000)]
Per kategori: [("Elektronik",22500000),("Aksesoris",2900000),("ATK",1000000)]
Per wilayah: [("Bandung",17400000),("Surabaya",7500000),("Jakarta",1500000)]
Laptop, Mouse: [("Elektronik",22500000),("Aksesoris",1500000)]
Trx Bandung > 1 juta: [1,5]
Lama+April = hitung ulang: True
Data asli: (5,26400000)

```

KASUS 4: SISTEM PAKAR REKOMENDASI

Organisasi membutuhkan sistem pakar yang memberi rekomendasi dari kondisi pengguna berdasarkan fakta, hubungan antar fakta, dan aturan yang disusun pakar. Aturan bisa saling terhubung, sehingga kesimpulan dari satu aturan dipakai untuk menarik kesimpulan berikutnya. Jika beberapa aturan terpenuhi, sistem harus mengembalikan semua kemungkinan jawaban, bukan hanya satu. Pakar harus bisa menambah atau mengubah fakta dan aturan tanpa membongkar mekanisme pencarian solusi, dan pengguna memperoleh kesimpulan dengan mengajukan query. Tantangannya adalah merancang representasi pengetahuan dan aturan, serta cara sistem memeriksa beberapa kemungkinan sampai menemukan jawaban.

Pendekatan utama: **Logic (SWI-Prolog)**.

Tabel 4. Analisis Kasus 4

Aspek Analisis	Jawaban
Karakteristik masalah	Masalahnya berupa penalaran berbasis aturan: dari kondisi pengguna, fakta, dan aturan buatan pakar, sistem memutuskan kondisi mana yang terpenuhi lalu rekomendasi apa yang sesuai. Sebagai contoh dipakai rekomendasi olahraga, dengan kondisi pengguna berupa BMI, tujuan latihan, dan keluhan sendi. Aturannya berantai, misalnya BMI minimal 25 menghasilkan kesimpulan "berat berlebih" yang dipakai lagi untuk menyimpulkan "sendi rentan", dan satu query bisa punya beberapa jawaban benar. Pakar bisa menambah atau mengubah fakta dan aturan kapan saja, sedangkan mekanisme pencarian solusinya harus tetap sama.
Konsep komputasi yang dominan	Setiap query diperlakukan sebagai tujuan yang harus dibuktikan dari fakta dan aturan lewat unifikasi dan backward chaining. Query rekomendasi(budi, 0) dicocokkan dengan kepala aturan, lalu dipecah menjadi subgoal butuh/2, olahraga/3, dan aman/2 yang dibuktikan dari kiri ke kanan. Subgoal yang punya beberapa klausa cocok meninggalkan choice point, dan Prolog menelusurinya secara depth-first: renang lolos, lari gagal di aman/2 karena sendi_rentan(budi) terbukti, lalu Prolog backtracking ke choice point terakhir dan mencoba sepeda_statis. findall/3 memaksa penelusuran ini berlanjut sampai semua cabang habis, sehingga semua rekomendasi terkumpul.
Pendekatan pemrograman yang dipilih	Dipakai paradigma Logic dari tabel slide, dengan cara berpikir "What conditions/rules hold?" dan abstraksi utama facts & rules. Bahasaanya Prolog dengan implementasi SWI-Prolog: unifikasi, backtracking, findall/3, dan assertz/1 sudah tersedia di bahasanya, dan SWI-Prolog menambahkan tabling untuk aturan yang saling memanggil. Basis pengetahuan pakar disimpan di berkas .pl berisi fakta dan aturan, lalu pengguna bertanya lewat query seperti ?- rekomendasi(sari, 0) . yang jawabannya keluar satu per satu saat menekan ; , atau sekaligus lewat findall/3.

Aspek Analisis	Jawaban
Alasan pemilihan berdasarkan karakteristiknya	Kebutuhan kasus punya padanan langsung di Prolog. Hubungan antar fakta ditulis sebagai predikat, misalnya <code>olahraga(renang, kardio, rendah)</code> yang menghubungkan olahraga dengan jenisnya dan dampaknya ke sendi. Kesimpulan berantai terjadi dengan sendirinya karena badan aturan boleh memanggil aturan lain, seperti <code>sendi_rentan/1</code> yang memakai hasil <code>berat_berlebih/1</code> , dan beberapa rekomendasi muncul lewat backtracking tanpa loop buatan sendiri. Mesin pencariannya bawaan Prolog, jadi pakar cukup menambah atau mengganti klausa di basis pengetahuan. Kalau ditulis imperatif di C, aturan yang sama tertanam di if-else bersarang dan urutan pengecekannya diatur manual, sehingga setiap aturan baru berarti mengubah alur program.
Contoh struktur/pseudocode	Kode dibagi menjadi fakta kondisi pengguna, fakta pakar tentang olahraga, dan aturan berantai (<code>berat_berlebih/1</code> , <code>sendi_rentan/1</code> , <code>butuh/2</code> , <code>aman/2</code>) yang dipakai <code>rekomendasi/2</code> . <code>tampil/1</code> mengumpulkan semua jawaban dengan <code>findall/3</code> lalu membuang jawaban ganda dengan <code>sort/2</code> , sedangkan <code>main/0</code> menjalankan query yang berhasil, yang gagal, dan query terbalik <code>rekomendasi(P, yoga)</code> , lalu menambah aturan serta fakta lewat <code>assertz/1</code> tanpa mengubah <code>rekomendasi/2</code> . Pada output, <code>renang</code> muncul dua kali untuk budi karena dua klausa <code>butuh/2</code> terpenuhi, dan setelah aturan baru masuk, budi juga mendapat <code>yoga</code> dan <code>pilates</code> . Potongan kodenya ada di Kode 4 di bawah tabel ini.
Kelebihan	Aturan pakar bisa ditulis hampir sama dengan kalimat aslinya, misalnya "BMI minimal 25 berarti berat berlebih" menjadi satu baris <code>berat_berlebih/1</code> , sehingga mudah diperiksa bersama pakar. Rantai kesimpulan dan pencarian semua jawaban sudah diurus mesin Prolog, jadi programmer cukup menulis fakta dan aturan. Satu predikat bisa menjawab beberapa bentuk pertanyaan: <code>rekomendasi(budi, O)</code> mencari olahraga untuk budi, sedangkan <code>rekomendasi(P, yoga)</code> mencari siapa yang cocok yoga dan menghasilkan <code>sari</code> . Pengetahuan baru cukup ditambahkan sebagai klausa, seperti aturan kelenturan pada contoh kode.
Keterbatasan	Prolog murni hanya mengenal terbukti atau gagal, padahal pengetahuan pakar sering berupa tingkat keyakinan seperti "cukup disarankan", sehingga bobot atau certainty factor harus dibuat sendiri. Kesimpulan yang punya dua jalur bukti menghasilkan jawaban ganda, seperti <code>renang</code> untuk budi, jadi hasilnya perlu dirapikan dengan <code>sort/2</code> . Pencarian depth-first juga bisa lambat saat aturan sudah ratusan, karena subgoal yang sama dikerjakan ulang, misalnya <code>berat_berlebih(budi)</code> dibuktikan tiga kali dalam satu query <code>rekomendasi(budi, O)</code> . Form isian, penyimpanan data, dan tampilan hasil lebih repot dibuat di Prolog, dan tidak banyak programmer yang terbiasa men-debug alur backtracking.

Aspek Analisis	Jawaban
Risiko terhadap kualitas software	Aturan baru dari pakar bisa membuat query tidak pernah selesai: jika ditambah <code>berat_berlebih(P) :- sendi_rentan(P)</code> , padahal sudah ada <code>sendi_rentan(P) :- berat_berlebih(P)</code> , query <code>sendi_rentan(dodi)</code> berputar tanpa henti, kecuali salah satu predikat di siklus itu diberi tabling (<code>:- table sendi_rentan/1</code>). Negation as failure dengan closed-world assumption membuat <code>dodi</code> direkomendasikan <code>lari</code> hanya karena keluhan lututnya tidak tercatat, bukan karena terbukti sehat. Urutan subgoal juga berpengaruh: jika <code>aman/2</code> dipanggil sebelum <code>butuh/2</code> , query <code>rekomendasi(P, lari)</code> kehilangan jawaban <code>dodi</code> , karena <code>sendi_rentan(P)</code> dijalankan saat <code>P</code> belum terikat dan gagal begitu ada satu pengguna bersendi rentan. Tanpa uji ulang setiap kali basis pengetahuan berubah, kesalahan seperti ini baru diketahui setelah sampai ke pengguna.
ISO/IEC 25010 yang relevan	Functional Suitability (functional correctness, functional completeness): setiap rekomendasi harus sesuai aturan pakar, dan semua olahraga yang aturannya terpenuhi harus muncul, bukan hanya jawaban pertama. Maintainability (modularity, modifiability, testability): basis pengetahuan terpisah dari mesin pencarian sehingga pakar bisa mengubah aturan sendiri, tetapi setiap perubahan perlu diuji ulang dengan kumpulan query yang hasilnya sudah diketahui, termasuk untuk menangkap aturan yang membentuk siklus. Performance Efficiency (time behaviour): backtracking pada ratusan aturan berantai bisa membuat satu query lambat dijawab. Safety (risk identification, fail safe): aturan seperti <code>sendi_rentan/1</code> harus mengenali pengguna yang berisiko cedera, dan data keluhan yang kosong seharusnya membuat sistem memilih olahraga berdampak rendah, bukan mengganggu pengguna sehat.
Apakah perlu kombinasi pendekatan?	Ya. Prolog dipakai untuk inti penalaran, yaitu basis pengetahuan dan pencarian jawaban, tetapi kurang cocok untuk tampilan dan penyimpanan data. Form isian kondisi pengguna dan halaman hasil dibuat event-driven dengan JavaScript: handler tombol kirim mengirim data ke backend. Backend object-oriented di Java menyimpan data pengguna dan riwayat rekomendasi ke basis data, lalu memanggil SWI-Prolog lewat library JPL: kondisi pengguna dimasukkan sebagai fakta <code>bmi/2</code> , <code>tujuan/2</code> , dan <code>keluhan/2</code> dengan <code>assertz/1</code> , kemudian query <code>rekomendasi/2</code> dijalankan dan hasilnya dikirim balik ke browser.

Kode 4. Sistem pakar rekomendasi olahraga di SWI-Prolog (Prolog, berkas rekomendasi.pl)

```
% Kasus 4: sistem pakar rekomendasi olahraga (SWI-Prolog)
:- initialization(main, main).
:- dynamic bmi/2, tujuan/2, keluhan/2, butuh/2, olahraga/3.

% Fakta kondisi pengguna, hasil isian form (masuk lewat assertz)
bmi(budi, 28).
bmi(sari, 21).
bmi(dodi, 22).
tujuan(budi, turun_berat).
tujuan(sari, turun_berat).
tujuan(sari, kelenturan).
tujuan(dodi, turun_berat).
keluhan(sari, nyeri_lutut).

% Fakta dari pakar: olahraga>Nama, Jenis, DampakKeSendi
olahraga(renang, kardio, rendah).
olahraga(lari, kardio, tinggi).
```

```

olahraga(sepeda_statis, kardio, rendah).
olahraga(yoga, kelenturan, rendah).

% Aturan pakar, kesimpulan satu aturan dipakai aturan berikutnya
berat_berlebih(P) :- bmi(P, B), B >= 25.
sendi_rentan(P) :- keluhan(P, nyeri_lutut).
sendi_rentan(P) :- berat_berlebih(P).
butuh(P, kardio) :- tujuan(P, turun_berat).
butuh(P, kardio) :- berat_berlebih(P).
butuh(P, kelenturan) :- tujuan(P, kelenturan).
% \+ artinya "tidak bisa dibuktikan", bukan "terbukti tidak"
aman(_, O) :- olahraga(O, _, rendah).
aman(P, O) :- olahraga(O, _, tinggi), \+ sendi_rentan(P).
rekomendasi(P, O) :- butuh(P, J), olahraga(O, J, _), aman(P, O).

% Query dari sisi pengguna
cek(G) :- (call(G) -> R = true ; R = false), format("?- ~w. ~w~n", [G, R]).
tampil(P) :-
    findall(O, rekomendasi(P, O), Semua), % backtracking sampai habis
    sort(Semua, Unik), % buang jawaban ganda
    format("~w: ~w~n unik: ~w~n", [P, Semua, Unik]).

main :-
    forall(bmi(U, _), tampil(U)),
    cek(sendi_rentan(budi)), % terbukti lewat berat_berlebih, bukan keluhan
    cek(rekomendasi(budi, lari)),
    findall(P, rekomendasi(P, yoga), Ps), % query dibalik: siapa cocok yoga
    format("?- rekomendasi(P,yoga). semua P: ~w~n", [Ps]),
    % pakar menambah aturan dan fakta, mesin pencarian tidak diubah
    assertz((butuh(X, kelenturan) :- sendi_rentan(X))),
    assertz(olahraga(pilates, kelenturan, rendah)),
    writeln('setelah pakar menambah aturan dan fakta:'),
    tampil(budi).

```

Dijalankan dengan perintah `swipl rekomendasi.pl`, outputnya:

```

budi: [renang,sepeda_statis,renang,sepeda_statis]
unik: [renang,sepeda_statis]
sari: [renang,sepeda_statis,yoga]
unik: [renang,sepeda_statis,yoga]
dodi: [renang,lari,sepeda_statis]
unik: [lari,renang,sepeda_statis]
?- sendi_rentan(budi). true
?- rekomendasi(budi,lari). false
?- rekomendasi(P,yoga). semua P: [sari]
setelah pakar menambah aturan dan fakta:
budi: [renang,sepeda_statis,renang,sepeda_statis,yoga,pilates]
unik: [pilates,renang,sepeda_statis,yoga]

```

KASUS 5: LAYANAN WEB DENGAN BANYAK PENGGUNA

Platform layanan digital menerima ribuan permintaan lewat web API pada waktu yang hampir bersamaan, mulai dari login, melihat data, transaksi, unggah file, sampai membaca database. Lama pemrosesan tiap permintaan berbeda karena ada yang menunggu database, ada yang menunggu layanan eksternal, dan ada yang butuh komputasi berat. Pada pemesanan, cek ketersediaan barang, simpan transaksi, pembaruan stok, pembayaran, dan notifikasi tidak semuanya harus berjalan berurutan. Sistem harus tetap responsif saat pengguna bertambah, tanpa konflik ketika beberapa proses mengubah data yang sama seperti stok, dan tanpa proses yang saling menunggu sampai layanan berhenti. Tantangannya adalah merancang pemrosesan dengan thread, proses, operasi asinkron, komunikasi antarproses, dan sinkronisasi.

Pendekatan utama: **Concurrent dengan operasi asinkron (Java 25).**

Tabel 5. Analisis Kasus 5

Aspek Analisis	Jawaban
Karakteristik masalah	Permintaan datang dari ribuan pengguna pada waktu yang berdekatan, dan lama prosesnya tidak sama: login dan melihat data cukup membaca database, pembayaran menunggu layanan eksternal, sedangkan sebagian permintaan butuh komputasi yang lama. Satu pemesanan pun terdiri dari beberapa langkah (cek ketersediaan barang, simpan transaksi, perbarui stok, bayar, kirim notifikasi) yang sebagian bisa berjalan bersamaan. Semua permintaan itu berbagi sumber daya yang sama, misalnya stok barang dan tabel transaksi, sehingga urutan eksekusi yang tidak diatur bisa menghasilkan data yang salah atau proses yang saling menunggu sampai layanan berhenti.
Konsep komputasi yang dominan	Konkurensi: banyak tugas berjalan dalam rentang waktu yang sama dengan eksekusi yang saling berselang. Sebagian besar waktunya habis untuk menunggu I/O (query database, respons gateway pembayaran), jadi selama satu permintaan menunggu, CPU dipakai permintaan lain lewat operasi asinkron. Stok barang adalah shared mutable state, sehingga cek stok lalu kurangi stok harus menjadi critical section yang atomik. Deadlock dicegah dengan memutus circular wait lewat urutan penguncian yang tetap atau <code>tryLock</code> berbatas waktu, supaya dua pesanan tidak saling menunggu kunci barang yang dipegang pesanan lain.
Pendekatan pemrograman yang dipilih	Paradigma Concurrent dari tabel slide, dengan cara berpikir "What happens simultaneously?" dan abstraksi processes/tasks/messages, ditambah operasi asinkron. Bahasanya Java 25: setiap permintaan berjalan di satu virtual thread (<code>newVirtualThreadPerTaskExecutor</code>), akses database dan gateway pembayaran memakai <code>CompletableFuture</code> dengan batas waktu, dan data bersama dijaga dengan <code>ReentrantLock</code> serta operasi atomik <code>ConcurrentHashMap</code> . Notifikasi dikirim lewat message queue seperti RabbitMQ ke layanan notifikasi yang berjalan sebagai proses terpisah. Go (goroutine dan channel) dan Erlang (proses ringan yang saling mengirim pesan) bisa mengekspresikan hal yang sama, tetapi Java dipilih karena framework web API dan driver database-nya matang.

Aspek Analisis	Jawaban
Alasan pemilihan berdasarkan karakteristiknya	Sebagian besar permintaan lebih lama menunggu database atau layanan eksternal daripada memakai CPU, jadi model satu permintaan satu thread cocok asalkan thread-nya murah. Virtual thread hanya butuh memori kecil dan dijadwalkan JVM di atas beberapa thread OS, sedangkan thread OS biasa masing-masing memesan stack 1 sampai 2 MB. Thread dipakai untuk permintaan di dalam satu proses server karena bisa berbagi connection pool dan cache, sedangkan notifikasi dan komputasi berat dipindah ke proses terpisah supaya crash di sana tidak menjatuhkan API. Langkah pemesanan yang lamanya berbeda cocok dengan <code>CompletableFuture</code> , dan konflik pada stok ditangani lock yang eksplisit.
Contoh struktur/pseudocode	<code>main</code> menjalankan 1000 permintaan, masing-masing di virtual thread sendiri: user 1 sampai 900 memesan buku dan pena (urutannya dibalik untuk user genap), user 901 sampai 1000 berebut 10 laptop, dan gateway dibuat lambat untuk user ke-300, 600, dan 900. <code>pesan</code> mengunci barang dalam urutan abjad, mengecek dan mengurangi stok dalam satu critical section, lalu menjalankan simpan transaksi dan pembayaran bersamaan dengan <code>CompletableFuture</code> yang dibatasi 300 ms, dan hanya pesanan yang berhasil masuk ke <code>antreanNotif</code> . Hasilnya 90 pesanan laptop ditolak tanpa stok minus, sedangkan 3 pesanan gagal bayar dibatalkan dan stoknya dikembalikan dengan <code>merge</code> yang atomik, sehingga sisa buku dan pena 103. Potongan kodenya ada di Kode 5 di bawah tabel ini.
Kelebihan	Server tetap responsif karena saat satu pesanan menunggu gateway pembayaran, permintaan lain seperti login dan melihat data tetap dilayani. Pada contoh kode, 1000 pesanan yang sebagian besar menunggu gateway 50 ms selesai dalam sekitar 330 ms, padahal kalau dikerjakan berurutan butuh lebih dari 45 detik. Dengan virtual thread, logika tiap permintaan tetap ditulis berurutan seperti kode biasa sehingga mudah dibaca dan stack trace-nya jelas, bukan rantai callback. Pesanan juga tetap selesai walaupun layanan notifikasi sedang lambat, karena notifikasi hanya dititipkan ke <code>antrean</code> .
Keterbatasan	Virtual thread hanya membantu permintaan yang banyak menunggu I/O; untuk komputasi yang lama, jumlah core CPU tetap menjadi batas, jadi pekerjaan itu perlu pool atau worker tersendiri. <code>ReentrantLock</code> pada contoh hanya berlaku di dalam satu JVM, padahal di produksi server API biasanya berjalan lebih dari satu instance, sehingga konflik stok antar server harus diselesaikan di database dengan transaksi atau <code>SELECT ... FOR UPDATE</code> . Lock per barang juga membuat pesanan untuk barang yang sama tetap antre satu per satu, jadi barang yang laris bisa menjadi titik macet. Ribuan virtual thread pun tetap berebut connection pool database yang jumlahnya terbatas.
Risiko terhadap kualitas software	Risiko terbesar adalah race condition: jika cek stok dan pengurangan stok tidak atomik, dua pengguna yang membeli laptop terakhir bisa sama-sama melihat stok 1, keduanya berhasil, dan stok menjadi -1. Deadlock juga nyata: ketika <code>sorted()</code> dihapus dari contoh kode, user ganjil mengunci buku lalu pena sementara user genap mengunci pena lalu buku, dan program macet di tiga kali percobaan. Timeout ke gateway membawa risiko lain, karena pembayaran yang dianggap gagal bisa saja sudah diproses, sehingga retry tanpa idempotency key bisa menagih pengguna dua kali. Bug seperti ini jarang muncul saat pengujian dengan sedikit pengguna dan baru terlihat ketika beban tinggi.

Aspek Analisis	Jawaban
ISO/IEC 25010 yang relevan	Performance Efficiency (time behaviour, resource utilization, capacity): waktu respons login dan pemesanan harus tetap rendah saat ribuan pengguna aktif, dan virtual thread menghemat memori dibanding satu thread OS per permintaan. Flexibility (scalability): ketika pengguna bertambah, kapasitas cukup dinaikkan dengan menambah instance server dan worker tanpa mengubah logika pemesanan. Reliability (availability, fault tolerance, recoverability): deadlock atau gateway yang lambat tidak boleh menghentikan layanan, sehingga dipakai urutan kunci dan timeout, sedangkan notifikasi yang tertunda tetap tersimpan di antrian sampai layanan notifikasi pulih. Functional Suitability (functional correctness): stok dan transaksi harus tetap benar walaupun banyak pesanan mengubahnya bersamaan, tanpa stok minus atau tagihan ganda.
Apakah perlu kombinasi pendekatan?	Ya. Server API memakai pendekatan concurrent dengan virtual thread untuk menangani permintaan, sedangkan notifikasi dibuat event-driven: setelah pembayaran lunas, server menerbitkan event "pesanan berhasil" ke message queue, lalu layanan notifikasi di proses terpisah menanganinya lewat handler, misalnya ditulis dengan Node.js. Komputasi yang lama, seperti memproses file yang diunggah, dikirim ke worker process lewat antrian yang sama supaya tidak menahan thread permintaan. Perhitungan total harga dan validasi pesanan ditulis gaya functional dengan data immutable seperti <code>record</code>, karena data yang tidak pernah diubah tidak perlu dikunci. Pengurangan stok antar server tetap dijaga oleh transaksi database.

Kode 5. Pemesanan serentak dengan virtual thread, lock, dan CompletableFuture di Java 25 (Java, berkas Pesanan.java)

```
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.locks.ReentrantLock;
import static java.util.concurrent.CompletableFuture.*;
import static java.util.concurrent.TimeUnit.MILLISECONDS;

public class Pesanan {
    static Map<String, Integer> stok = new ConcurrentHashMap<>()
        .of("buku", 1000, "pena", 1000, "laptop", 10);
    static Map<String, ReentrantLock> kunci = new ConcurrentHashMap<>();
    static BlockingQueue<String> antrianNotif = new LinkedBlockingQueue<>();
    static String pesan(int id, List<String> barang) {
        varurut = barang.stream().sorted().map(b -> kunci.computeIfAbsent(
            b, k -> new ReentrantLock())).toList();
        urut.forEach(ReentrantLock::lock); // urut abjad, cegah deadlock
        try { // cek ketersediaan dan kurangi stok harus satu langkah
            if (barang.stream().anyMatch(b -> stok.get(b) < 1))
                return "stok habis";
            barang.forEach(b -> stok.merge(b, -1, Integer::sum));
        } finally { urut.forEach(ReentrantLock::unlock); }
        // simpan transaksi (database) dan bayar (gateway) jalan bersamaan
        var simpan = supplyAsync(() -> "tersimpan",
            delayedExecutor(30, MILLISECONDS));
        var bayar = supplyAsync(() -> "lunas",
            delayedExecutor(id % 300 == 0 ? 900 : 50, MILLISECONDS)
                .orTimeout(300, MILLISECONDS)); // batas tunggu gateway
        boolean ok = allOf(simpan, bayar).handle((v, e) -> e == null).join();
        if (!ok) { // gateway terlalu lama: pesanan batal, stok dikembalikan
```

```

        barang.forEach(b -> stok.merge(b, 1, Integer::sum)); // atomik
        return "gagal bayar";
    }
    antreanNotif.add("user" + id); // diambil layanan notifikasi terpisah
    return "berhasil";
}

public static void main(String[] args) {
    var rekap = new ConcurrentHashMap<String, Integer>();
    var pool = Executors.newVirtualThreadPerTaskExecutor();
    for (int i = 1; i <= 1000; i++) { // 1 permintaan = 1 virtual thread
        int id = i;
        var barang = id > 900 ? List.of("laptop") : id % 2 == 0
            ? List.of("pena", "buku") : List.of("buku", "pena");
        pool.submit(() -> rekap.merge(pesan(id, barang), 1, Integer::sum));
    }
    pool.close(); // menunggu semua permintaan selesai
    System.out.println(new TreeMap<>(rekap));
    System.out.println("stok akhir: " + new TreeMap<>(stok));
    System.out.println("antrean notifikasi: " + antreanNotif.size());
}
}

```

Dijalankan dengan perintah `java Pesanan.java`, outputnya:

```

{berhasil=907, gagal bayar=3, stok habis=90}
stok akhir: {buku=103, laptop=0, pena=103}
antrean notifikasi: 907

```

RANGKUMAN

Tabel 6. Pendekatan Utama Setiap Kasus

Kasus	Pendekatan utama	Perlu kombinasi?
1. Sistem Keranjang Belanja E-Commerce	Object-Oriented (Java)	Ya.
2. Sistem Informasi Rumah Sakit	Object-Oriented (Java)	Ya.
3. Analisis Data Penjualan	Functional (Haskell)	Ya.
4. Sistem Pakar Rekomendasi	Logic (SWI-Prolog)	Ya.
5. Layanan Web dengan Banyak Pengguna	Concurrent dengan operasi asinkron (Java 25)	Ya.

Dari kelima kasus terlihat bahwa pendekatan dipilih dari bentuk masalahnya, bukan dari bahasa yang paling dikenal. Sistem informasi rumah sakit yang penuh entitas dan hak akses paling pas dengan object-oriented, analisis penjualan yang isinya transformasi data paling pas dengan functional, sistem pakar yang berisi fakta dan aturan paling pas dengan logic programming, dan layanan web dengan ribuan permintaan paling pas dengan concurrent programming. Keranjang belanja adalah kasus yang paling campuran, jadi OOP dipakai sebagai kerangka lalu dibantu fungsi murni untuk perhitungan checkout dan penguncian untuk pemotongan stok. Kelima tabel juga menunjukkan bahwa tidak ada kasus yang cukup memakai satu paradigma saja: bagian inti memakai pendekatan utamanya, sedangkan bagian pinggir seperti tampilan, penyimpanan data, dan notifikasi memakai pendekatan lain yang lebih cocok.